Mansoura University
Faculty of Computers and Information
Department of Information Technology
Second Semester- 2025-2026

[UNI311T]Software Reengineering

Grade: 4 SW

BY: Hala Ahmed

Outline

- Evolution versus Maintenance
  - Software Evolution
  - Software Maintenance
- Software Evolution Models and Processes
- Reengineering

Objectives

- **The objectives of this chapter** are to explain why software evolution is such an important part of software engineering and to describe the challenges of maintaining a large base of software systems, developed over many years. When you have read this chapter, you will:

  - **understand that software systems** have to adapt and evolve if they are to remain useful and that software change and evolution should be considered as an integral part of software engineering;

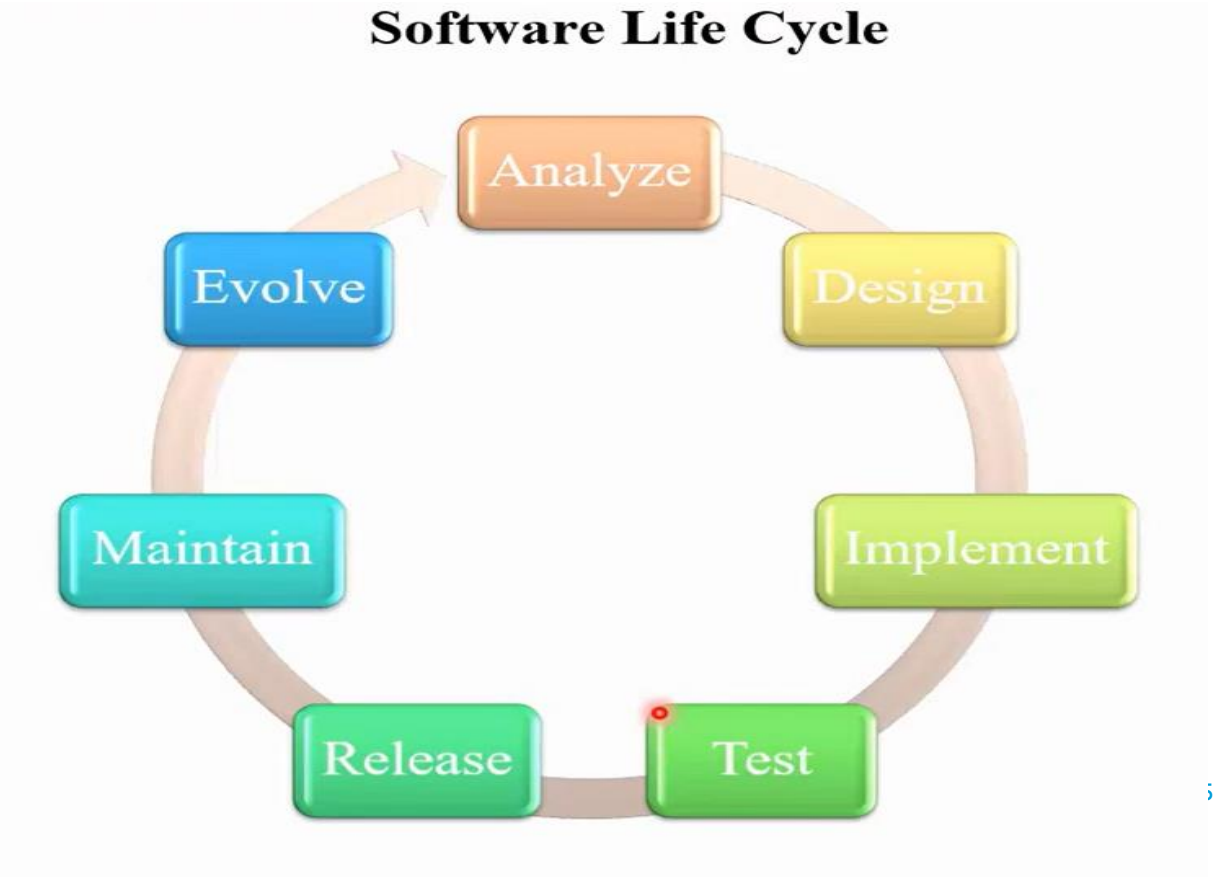
  - **understand what is meant by legacy systems** and why these systems are important to businesses;

Objectives

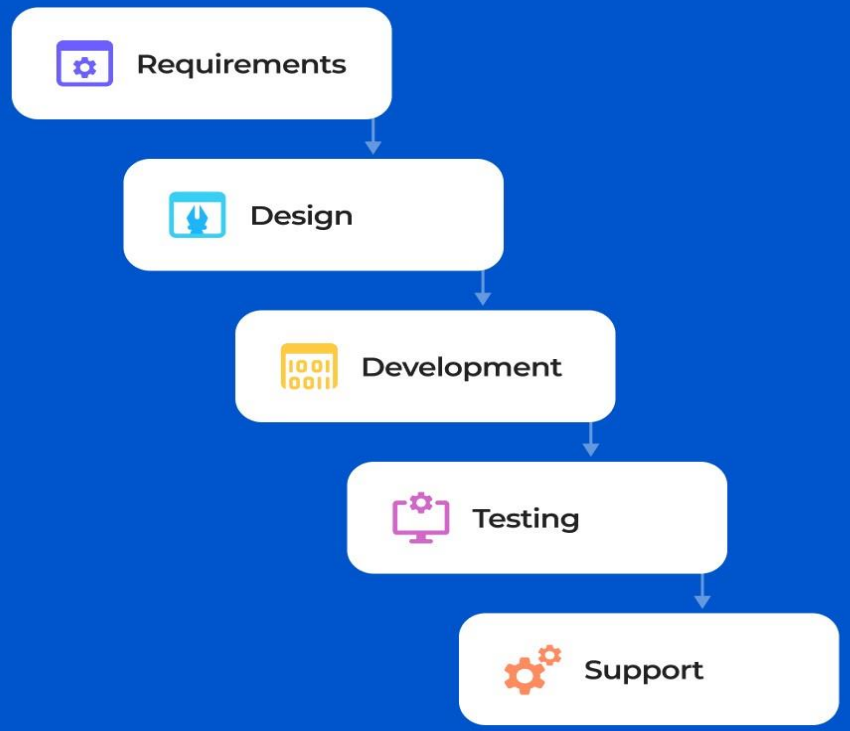
- **understand how legacy** systems can be assessed to decide whether they should be scrapped, maintained, reengineered, or replaced; have learned about different types of software maintenance and the factors that affect the costs of making changes to legacy software systems.

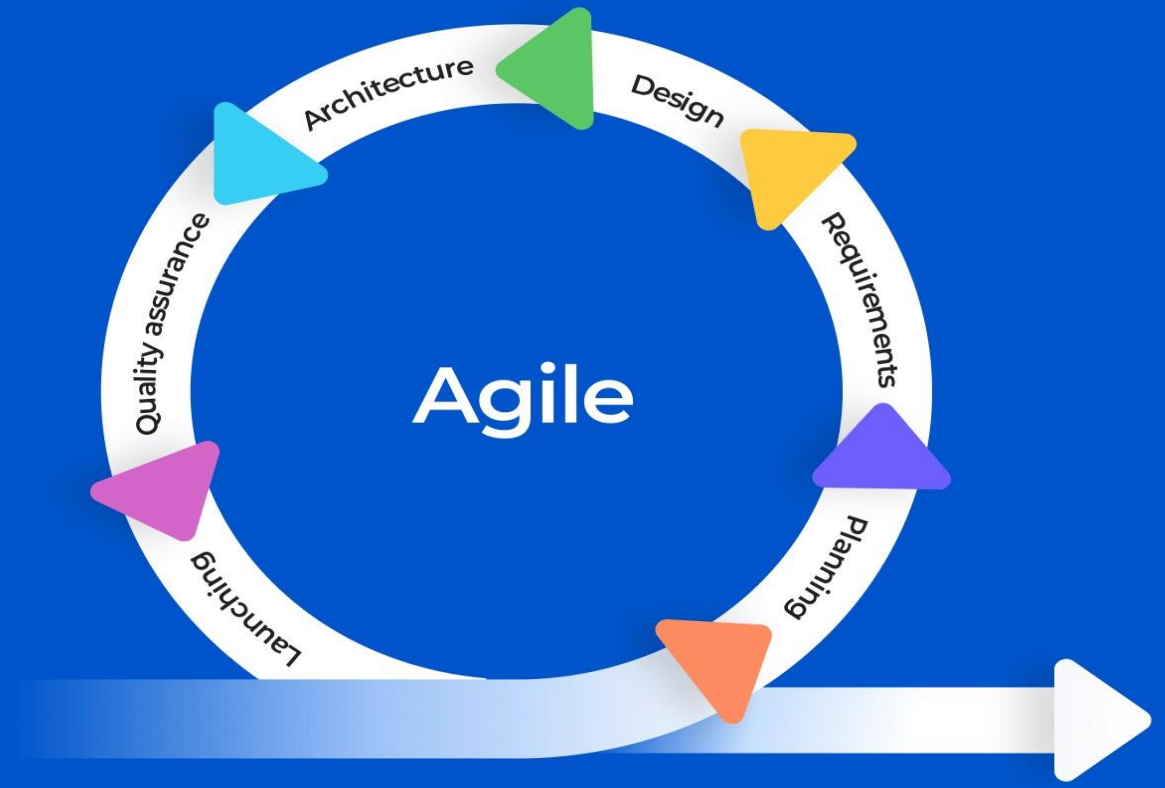
Software engineering

- **Software engineering:** is defined as a process of analyzing user requirements and then designing, building, and testing software application which will satisfy those requirements.



Waterfall and Agile models comparison





Software engineering(Cont.)

- A software product is judged by **how easily it can be used by the end-user** and the features it offers to the user.
- An application must **score in the following areas**:-

1. **Operational:**

   This tells how good a software works on operations like budget , usability, efficiency, correctness ,functionality , dependability , security and safety.

2. **Transitional:**

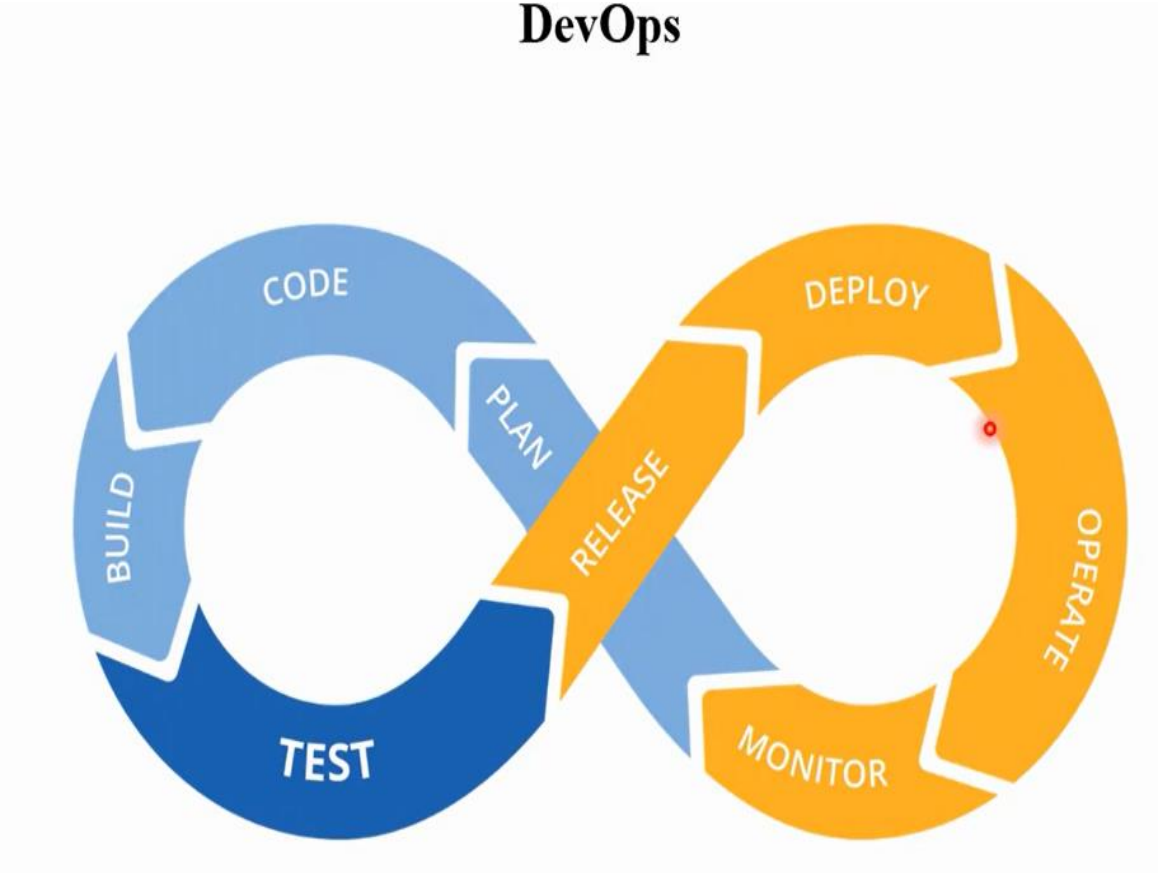
   Transitional is important when an application is shifted from one platform to another. So, portability, reusability and adaptability come in this area.

3. **Maintenance:**

   This specifies how good a software works in the changing environment. Modularity, maintainability, flexibility and scalability come in maintenance part.

DEVOPS

- **DevOps** is a set of practices that combines **software development** (*Dev*) and **IT operations** (*Ops*).
- It aims to shorten the **systems development life cycle** and provide **continuous delivery** with high **software quality**.



Evolution Versus Maintenance

- The terms **evolution** and **maintenance** **are used interchangeably.**
- However there is a semantic difference.
- **Lowell Jay Arthur distinguish the two terms as follows:**
  - **"Software maintenance** means to preserve from failure or decline."
  - **"Software evolution** means a continuous change from lesser, simpler, or worse state to a higher or better state."
- **Keith H. Bennett and Lie Xu use the term:**
  - "**maintenance** for all post-delivery support and evolution to those driven by changes in requirements."

Evolution Versus Maintenance (Cont.)

- **Maintenance** is considered to be **set of planned activities** whereas **evolution** concern whatever **happens to a system over time**.
- **Mehdi Jazayer's view on software evolution:**

> "Over time what **evolves is not the software** but our knowledge about a particular type of software."

Outline

- Evolution versus Maintenance
  - **Software Evolution**
  - Software Maintenance
- Software Evolution Models and Processes
- Reengineering

Software Evolution

- In 1965, **Mark Halpern** used the term **evolution** to define the **dynamic growth of software**.
- The term evolution in relation to application systems took gradually in the 1970s.
- **Lehman and his collaborators** from IBM are generally credited with pioneering the research field of software evolution.
- Lehman formulated a set of observations that he **called laws of evolution**.
- These laws are the results of **studies of the evolution of large-scale proprietary** or **closed source system (CSS).**
- The laws concern what Lehman called **E-type systems**:

"**Monolithic systems produced by a team within an organization that solve a real world problem and have human users**."

Software Evolution: Laws of Lehman

- **Continuing change (1st)** – A system will become progressively less satisfying to its user over time, unless it is continually adapted to meet new needs.

- **Increasing complexity (2nd)** – A system will become progressively more complex, unless work is done to explicitly reduce the complexity.

- **Self-regulation (3rd)** – The process of software evolution is self regulating with respect to the distributions of the products and process artifacts that are produced.

- **Conservation of organizational stability (4th)** – The average effective global activity rate on an evolving system does not change over time, that is the average amount of work that goes into each release is about the same.

Software Evolution: Laws of Lehman

- **Conservation of familiarity (5th) –** The amount of new content in each successive release of a system tends to stay constant or decrease over time.

- **Continuing growth (6th) –** The amount of functionality in a system will increase over time, in order to please its users.

- **Declining quality (7th) –** A system will be perceived as losing quality over time, unless its design is carefully maintained and adapted to new operational constraints.

- **Feedback system (8th)** – Successfully evolving a software system requires recognition that the development process is a multi-loop, multi-agent, multi-level feedback system.

Software Evolution: FOSS System

- In circa 1988, **Pirzada pointed out the differences** between the evolution of the Unix OS and system studied by Lehman
- Pirzada **argued that the differences** in **academic and industrial** s/w development could lead to a differences in the evolutionary pattern.
- In circa 2000, empirical study of free and open source software (FOSS) evolution was conducted by Godfrey and Tu.
- They found that the growth trends from 1994-1999 for the evolution of FOSS Lunix OS to be super-linear, that is greater than linear.

Software Evolution: FOSS System(Cont.)

- Robles and his collaborator concluded that Lehman's laws, 3, 4, and 5 are not fitted to large scale FOSS system such as Linux.

> "**FOSS** is made available to the general public with either relaxed or non-existent intellectual property restrictions. The free emphasizes the freedom to modify and redistribute under the terms of the original license while open emphasizes the accessibility to the source code."

Outline

- Evolution versus Maintenance
  - Software Evolution
  - **Software Maintenance**
- Software Evolution Models and Processes
- Reengineering

Software Maintenance

- There will always be **defects in the delivered software application** because **software defect removal** and **quality control** are not perfect.
- Therefore, software maintenance is needed to repair these defects in the released software.
- E. Burton Swanson defined **three type of software maintenance**:

  Corrective, Adaptive & Perfective.

- It is based on the intent of the developer towards the system.

Software Maintenance

- Swanson definition was later updated in the standard for software engineering – ISO/IEC 14764.

- Introduced a fourth category called **preventive maintenance**.

- Some researchers and developers view **preventive maintenance** as a subset of **perfective maintenance**.

Software Maintenance

- Kitchenham described **maintenance modifications in a hierarchical** ways based on the concept of activity:
  - **Activities to make corrections:** If there are **discrepancies between the expected behavior of a system and the actual behavior**, then some activities are performed to eliminate or reduce the discrepancies.
  - **Activities to make enhancements:** A number of activities are performed to implement a change to the system, thereby changing the behavior or implementation of the system.
    - **This category is subdivided into three types:**
      1. enhancements that change existing requirement,
      2. enhancements that add new system requirements, and
      3. enhancements that change the implementation but not the requirements.

Software Maintenance

- Chapin et al. expanded the typology of Swanson into an evidence-based classification of **12** different types of software maintenance:
  - ✓ Training
  - ✓ Evaluate
  - ✓ Updative
  - ✓ Preventive
  - ✓ Adaptive
  - ✓ Corrective
  - ✓ Consultive
  - ✓ Reformative
  - ✓ Groomative
  - ✓ Performance
  - ✓ Reductive
  - ✓ Enhancive

Software Maintenance: COTS

- The major differences between component-based software systems (CBS) and custom-built software systems:
  - Skills of system maintenance teams.
  - Infrastructure and organization.
  - COTS maintenance cost.
  - Larger user community.
  - Modernization.
  - Split maintenance function.
  - More complex planning.

Outline

- Evolution versus Maintenance
  - Software Evolution
  - Software Maintenance
  - **Software Evolution Models and Processes**
- Reengineering

Software Evolution Models and Processes

- **Software maintenance** have its own **software maintenance life cycle (SMLC) model**.
- **Three popular SMLC models:**
  - Staged model of maintenance and evolution.
  - Iterative models.
  - Change mini-cycle models.

Software Evolution Models and Processes

Software Maintenance Standards

- IEEE and ISO have both addressed s/w maintenance processes.
- IEEE/EIA 1219 and ISO/IEC 14764 as a part of ISO/IEC12207 (life cycle process).
- IEEE/EIA 1219 organizes the maintenance process in seven phases:
  - problem identification, analysis, design, implementation, system test, acceptance test and delivery.
- ISO/IEC 14764 describes s/w maintenance as an iterative process for managing and executing software maintenance activities.

Software Evolution Models and Processes

- **The activities** which comprise the **maintenance process are**:
  - process implementation, problem and modification analysis, modification implementation, maintenance review/acceptance, migration and retirement.
- Each of these maintenance activity is made up of tasks.
- Each task describes a specific **action with inputs and outputs**.

Software Evolution Models and Processes

Software Configuration Management

- **SCM** is the discipline of **managing** and **controlling change** in the evolution of software system.
- It ensures that the released software is **not contaminated** by **uncontrolled or unapproved changes**.
- An **SCM** system has four different elements, each element addressing a distinct user need as follows:
  - Identification of software configurations.
  - Control of software configurations.
  - Auditing software configurations.
  - Accounting software configuration status.

Outline

- Evolution versus Maintenance
  - Software Evolution
  - Software Maintenance
- Software Evolution Models and Processes
- **Reengineering**

Software Re-engineering

- **Software Re-engineering** is a process of software development which is done to **improve the maintainability** of a software system.
- **Re-engineering** is the **examination and alteration** of a system to reconstitute it in **a new form.**
- This **process encompasses** a combination of **sub-processes** like **reverse engineering**, **forward engineering**, **reconstructing** etc.

Re-engineering is the reorganizing and **modifying existing software** systems to make them **more maintainable**.

Reengineering

- **Reengineering** is done to **transform an existing** "**lesser or simpler**" system into a **new "better"** system.
- Reengineering is the **examination**, **analysis and restructuring** of an existing software system to reconstitute it in **a new form**, and the subsequent implementation of the new form.
- Chikofsky and Cross II defines reengineering as:

> **"the examination and alteration of a subject system to reconstitute it in a new form and the subsequent implementation of the new form."**

Reengineering

- Jacobson and Lindstorm defined following formula:

  Reengineering = Reverse engineering + Δ + Forward engineering.

- **Reverse engineering** is the activity of defining a more abstract, and easier to understand, representation of the system
- The core of reverse engineering is the process of examination, not a process of change, therefore it does not involve changing the software under examination.
- The third element "**forward engineering,**" is the traditional process of moving from high-level abstraction and logical, implementation-independent designs to the physical implementation of the system.
- The second element Δ captures alteration that is change of the system.

Advantages of Re-engineering:

- **Reduced Risk:** As the software is already existing, the risk is less as compared to new software development. Development problems, staffing problems and specification problems are the lots of problems which may arise in new software development.
- **Reduced Cost:** The cost of re-engineering is less than the costs of developing new software.
- **Revelation of Business Rules:** As a system is re-engineered , business rules that are embedded in the system are rediscovered.
- **Better use of Existing Staff:** Existing staff expertise can be maintained and extended accommodate new skills during re-engineering.

Disadvantages of Re-engineering

- Practical limits to the extent of re-engineering.
- Major architectural changes or radical reorganizing of the systems data management has to be done manually.
- Re-engineered system is not likely to be as maintainable as a new system developed using modern software Re-engineering methods.

Thank you